

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Колледж программирования и кибербезопасности

ОТЧЁТ

по практическому занятию № 1

«Отладка программы»

по междисциплинарному курсу МДК.01.02
«Поддержка и тестирование программных модулей»

Вариант 27

Специальность 09.02.07 «Информационные системы и программирование»

Выполнил
студент группы ПКС-33 _____ А. Шевырин

Проверила
преподаватель _____ М. Л. Мымрина

Москва 2026

СОДЕРЖАНИЕ

1 Цель работы.....3
2 Постановка задачи.....3
3 Средства разработки.....3
4 Описание программы.....4
4.1 Структура программы.....4
4.2 Алгоритм решения.....5
5 Отладка программы.....5
5.1 Настройка отладки.....5
5.2 Задание точек останова.....6
5.3 Запуск отладчика.....7
5.4 Пошаговое выполнение.....8
5.5 Проверка значений переменных и свойств объектов.....10
5.6 Установка контрольных значений.....12
5.7 Просмотр стека вызовов.....14
5.8 Условная точка останова.....14
5.9 Шаг с выходом и вычисление выражений.....16
5.10 Завершение работы программы.....16
6 Тестирование программы.....18
Заключение.....19
Приложение А (обязательное) Текст программы.....20
Приложение Б (обязательное) Файл настройки отладки.....22

1 Цель работы

Научиться максимально использовать средства отладки интегрированной среды разработки: задавать точки останова, выполнять программу по шагам, проверять значения переменных и свойств объектов, устанавливать контрольные значения, просматривать стек вызовов и настраивать параметры отладки под нужды автора программы.

2 Постановка задачи

Вариант 27. В массиве записана информация о стоимости 30 видов товара. Определить стоимость двух самых дорогих видов товара.

Необходимо разработать программу для задачи своего варианта и использовать при её отладке следующие методы:

1) задание точек останова;

2) реализация пошагового прохода;

3) проверка значений переменных и свойств объектов;

4) установка контрольных значений;

5) просмотр стека вызовов;

6) настройка отладки для нужд автора программы.

3 Средства разработки

Программа написана на языке C# для платформы .NET 8.0 (пакет SDK 8.0.426) в виде консольного приложения. Для разработки и отладки использовалась среда Visual Studio Code (сборка code-server 4.108) с расширением C# и отладчиком .NET (netcoredbg). Среда поддерживает те же средства отладки, что описаны в методических указаниях для Visual Studio, и те же сочетания клавиш: F5 – начать или продолжить отладку, F9 – точка останова, F10 – шаг с обходом, F11 – шаг с заходом, Shift+F11 – шаг с выходом, Ctrl+Shift+F5 – перезапуск, Shift+F5 – остановка отладки.

Окна отладчика имеют те же назначения, что и в Visual Studio: «Переменные» (раздел Locals – аналог окна «Локальные»), «Контрольное значение», «Стек вызовов», «Точки останова» и «Консоль отладки».

4 Описание программы

4.1 Структура программы

Вид товара описывается классом `Product` с двумя свойствами только для чтения: `Name` (название) и `Price` (стоимость, тип `decimal`). Метод `ToString` возвращает строку вида «Товар 8 – 49384.08 руб.». Использование объектов со свойствами позволяет при отладке проверять не только значения простых переменных, но и свойства объектов.

Логика программы размещена в классе `Program`. Состав методов приведён в таблице 1.

Таблица 1 – Методы класса Program

Метод	Назначение
Main	Точка входа. Разбирает аргумент командной строки, получает массив товаров, выводит его и результат поиска
GenerateProducts	Заполняет массив из 30 товаров случайными ценами от 100.00 до 50 000.00 руб. (зерно генератора задаётся аргументом, по умолчанию 27)
ReadProducts	Вводит 30 цен с клавиатуры; при нечисловом или отрицательном значении повторяет запрос
PrintProducts	Выводит номера и стоимости всех товаров
FindTwoMostExpensive	Находит за один проход индексы двух самых дорогих товаров и возвращает их через out-параметры first и second

Режим работы задаётся аргументом командной строки: число – зерно генератора случайных цен, `--manual` – ввод цен с клавиатуры. Это позволяет при отладке получать один и тот же воспроизводимый массив и не вводить 30 значений при каждом запуске. Полный текст программы приведён в приложении А.

4.2 Алгоритм решения

Для поиска двух максимальных значений массив не сортируется: достаточно одного прохода, при котором хранятся индекс самого дорогого товара `first` и индекс второго по стоимости товара `second`:

1) перед циклом `first = 0`, `second = -1` (второй максимум ещё не найден);

2) для каждого товара с индексом i от 1 до 29 берётся его цена price;

3) если price больше цены товара first, то прежний максимум становится вторым (second = first), а новым максимумом становится текущий товар (first = i);

4) иначе, если второй максимум ещё не найден или price больше цены товара second, то second = i.

Сложность алгоритма – O(n), исходный массив не изменяется. Если два разных вида товара имеют одинаковую максимальную стоимость, программа выводит обе одинаковые цены, так как это разные виды товара.

5 Отладка программы

5.1 Настройка отладки

Параметры запуска отладчика задаются в файле .vscode/launch.json (аналог вкладки «Отладка» в свойствах проекта Visual Studio). Для нужд программы созданы две конфигурации (рисунок 1):

– «Отладка: вариант 27 (seed = 27)» – передаёт программе аргумент 27, поэтому при каждом запуске формируется один и тот же массив цен; вывод направляется в консоль отладки, включён режим «Только мой код» (justMyCode), задана переменная среды DOTNET_ENVIRONMENT;

– «Отладка: ввод цен с клавиатуры» – передаёт аргумент --manual, запускает программу во встроенном терминале (для ввода с клавиатуры) и останавливается на первой строке Main (stopAtEntry).

Перед каждым запуском автоматически выполняется задача сборки build (параметр preLaunchTask), поэтому отладчик всегда работает с

актуальной версией кода. Ещё один вид настройки – условная точка останова – рассмотрен в подразделе 5.8.

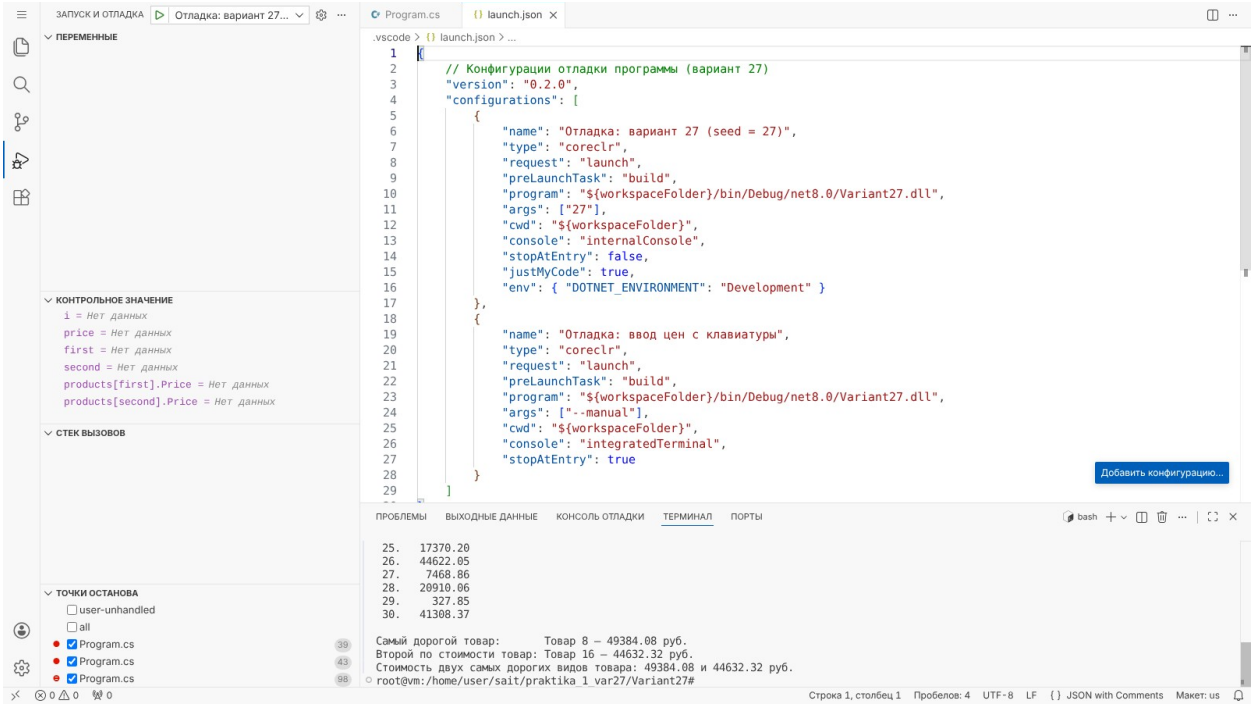


Рисунок 1 – Конфигурации отладки в файле launch.json

5.2 Задание точек останова

Точки останова установлены клавишей F9 (или щелчком слева от номера строки) на трёх строках (рисунок 2):

– строка 39 – получение массива товаров в методе Main;

– строка 43 – вызов метода поиска FindTwoMostExpensive;

– строка 98 – сравнение цены текущего товара с максимальной внутри цикла поиска.

Установленные точки отображаются красными кружками слева от номеров строк и перечислены в окне «Точки останова», где их можно временно отключить флажком, не удаляя.

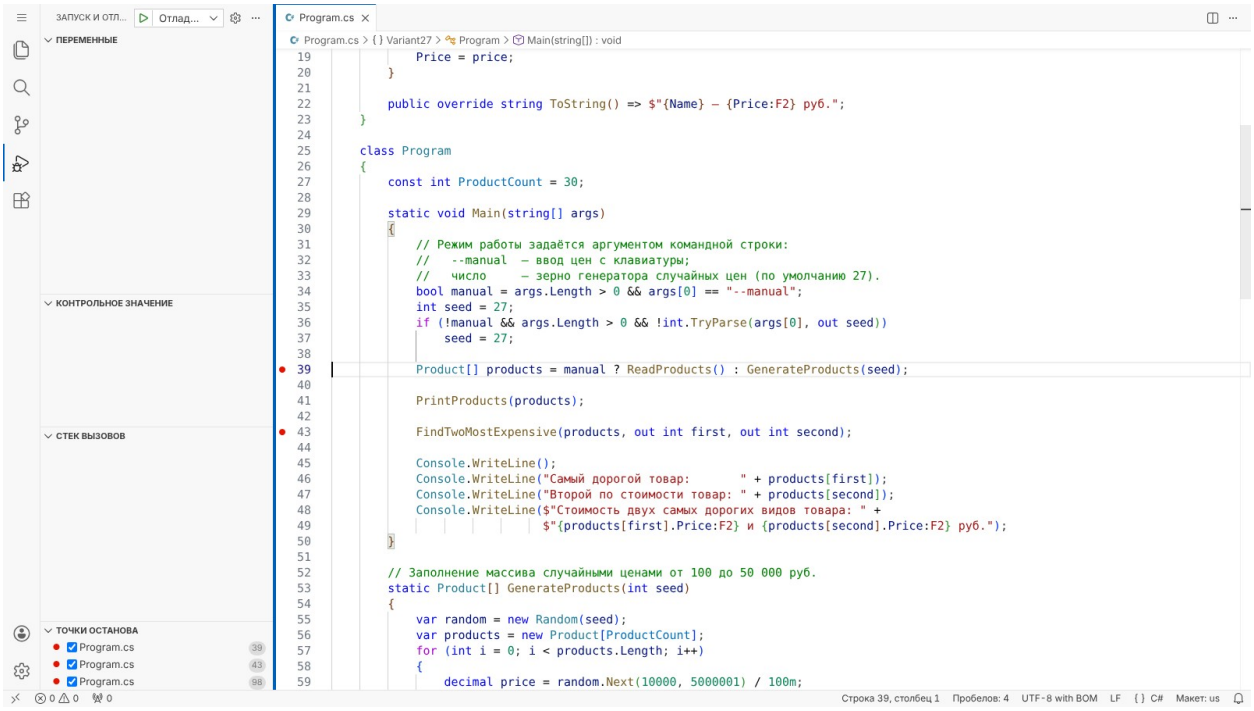


Рисунок 2 – Установленные точки останова

5.3 Запуск отладчика

Отладка запущена клавишей F5. После сборки программа остановилась на первой встреченной точке останова – строке 39 (рисунок 3). Жёлтая стрелка показывает оператор, который будет выполнен следующим. В окне «Переменные» видны локальные переменные метода Main: args = {string[1]}, manual = false, seed = 27 (значение получено из аргумента командной строки, заданного в launch.json), products = null – массив ещё не создан. Значения переменных также выводятся прямо в редакторе справа от строк кода.

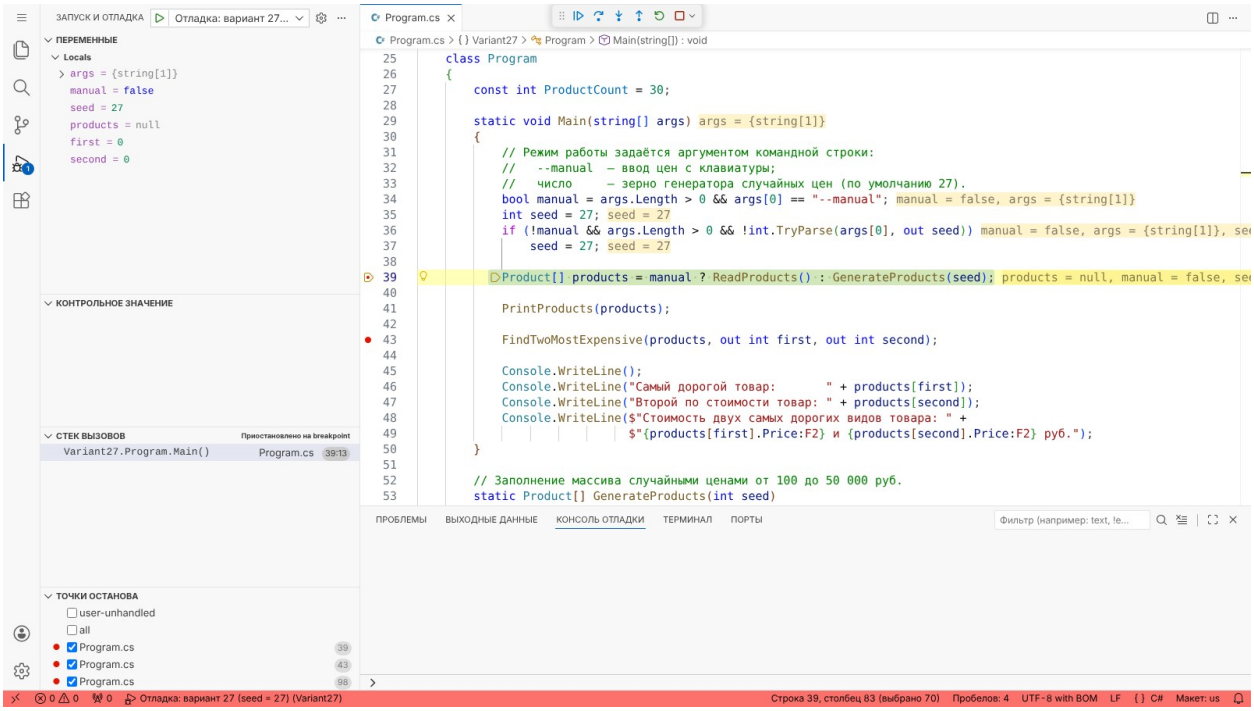


Рисунок 3 – Остановка на точке останова в строке 39

5.4 Пошаговое выполнение

Командой «Шаг с заходом» (F11) выполнен переход внутрь метода GenerateProducts, после чего несколькими командами «Шаг с обходом» (F10) выполнена первая итерация цикла заполнения массива (рисунок 4). В окне «Стек вызовов» появилась вторая строка: текущий метод GenerateProducts вызван из Main. Счётчик цикла i = 1, то есть первый элемент массива уже создан.

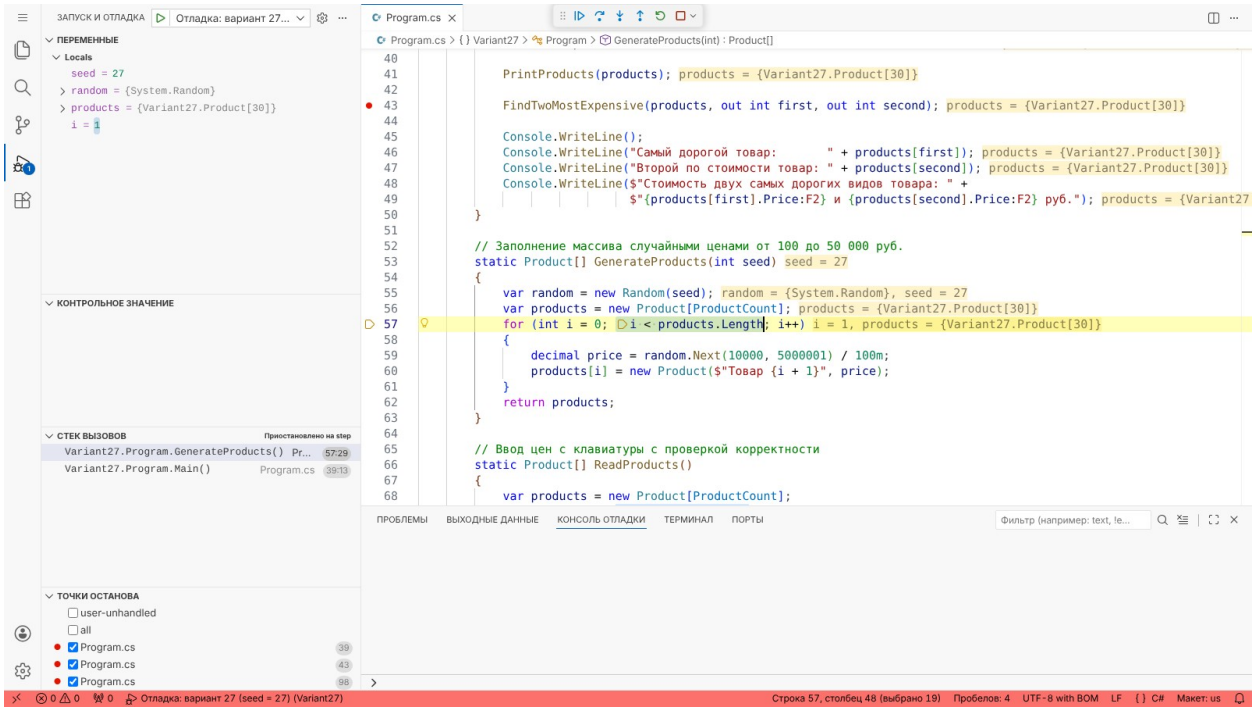


Рисунок 4 – Шаг с заходом в метод GenerateProducts и выполнение цикла по шагам

Чтобы не проходить по шагам все 30 итераций, выполнен «Шаг с выходом» (Shift+F11) – отладчик завершил метод GenerateProducts и вернулся в Main. Далее метод PrintProducts выполнен командой «Шаг с обходом» (F10): отладчик не заходит внутрь метода, но метод выполняется полностью, и в консоли отладки появился список цен всех 30 товаров (рисунок 5). Желтая стрелка переместилась на строку 43, где находится вторая точка останова.

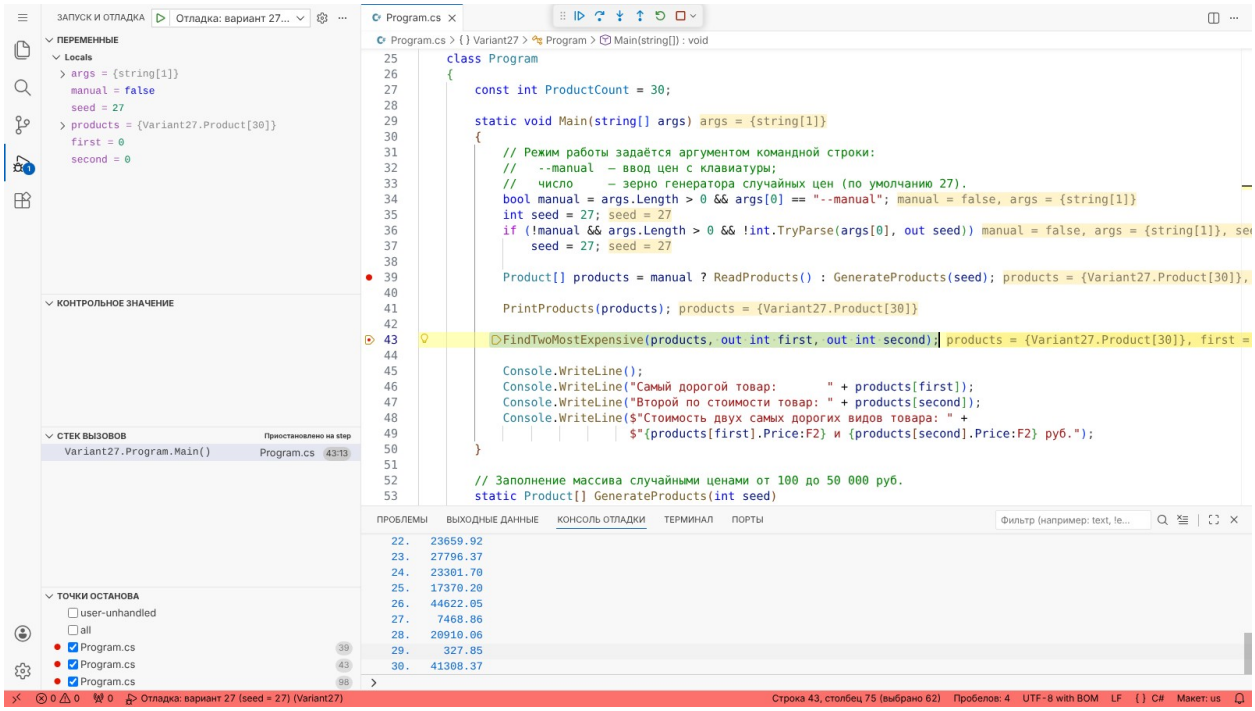


Рисунок 5 – Результат шага с обходом метода PrintProducts

5.5 Проверка значений переменных и свойств объектов

Во время пошагового прохода метода GenerateProducts в окне «Переменные» развёрнут массив products и его первый элемент (рисунок 6). Элемент [0] – объект класса Product со свойствами Name = "Товар 1" и Price = 41603.17, остальные элементы пока равны null, так как цикл выполнил только одну итерацию. Это подтверждает, что массив заполняется по порядку и объекты создаются корректно.

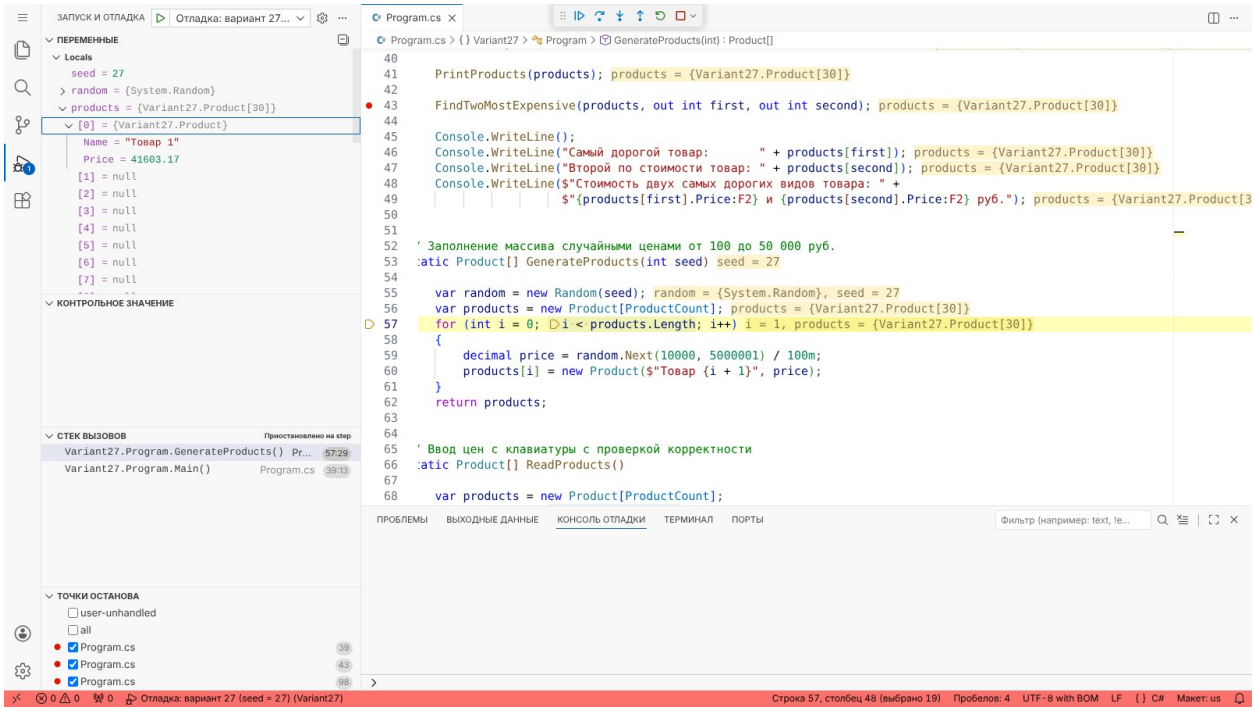


Рисунок 6 – Свойства объекта Product в окне «Переменные»

После заполнения массива значение переменной проверено с помощью подсказки по данным: при наведении указателя мыши на переменную products в строке 41 появилось окно со всеми элементами массива (рисунок 7). В подсказке раскрыт элемент [7]: Name = "Товар 8", Price = 49384.08 – как показала дальнейшая отладка, это самый дорогой товар.

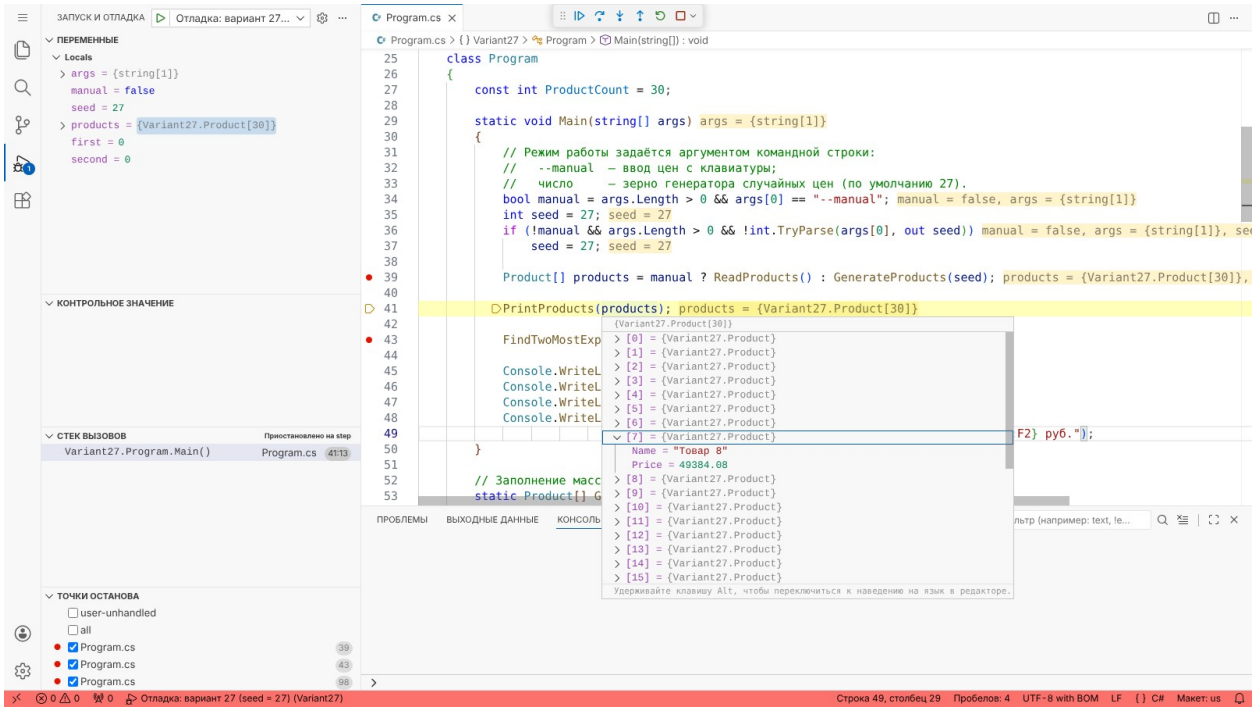


Рисунок 7 – Подсказка по данным для массива products

5.6 Установка контрольных значений

В окно «Контрольное значение» добавлены выражения, которые нужно отслеживать при работе алгоритма поиска: i, price, first, second, products[first].Price и products[second].Price. Затем выполнен шаг с заходом в метод FindTwoMostExpensive и продолжение (F5) до точки останова в строке 98 (рисунок 8).

На первой итерации i = 1, price = 37007.44, first = 0, second = -1. Сначала в качестве шестого выражения было задано выражение с условным оператором second >= 0 ? products[second].Price : -1, однако отладчик .NET не вычисляет условный оператор в контрольных значениях и выводит ошибку (последняя строка окна на рисунке 8). Поэтому выражение заменено на products[second].Price.

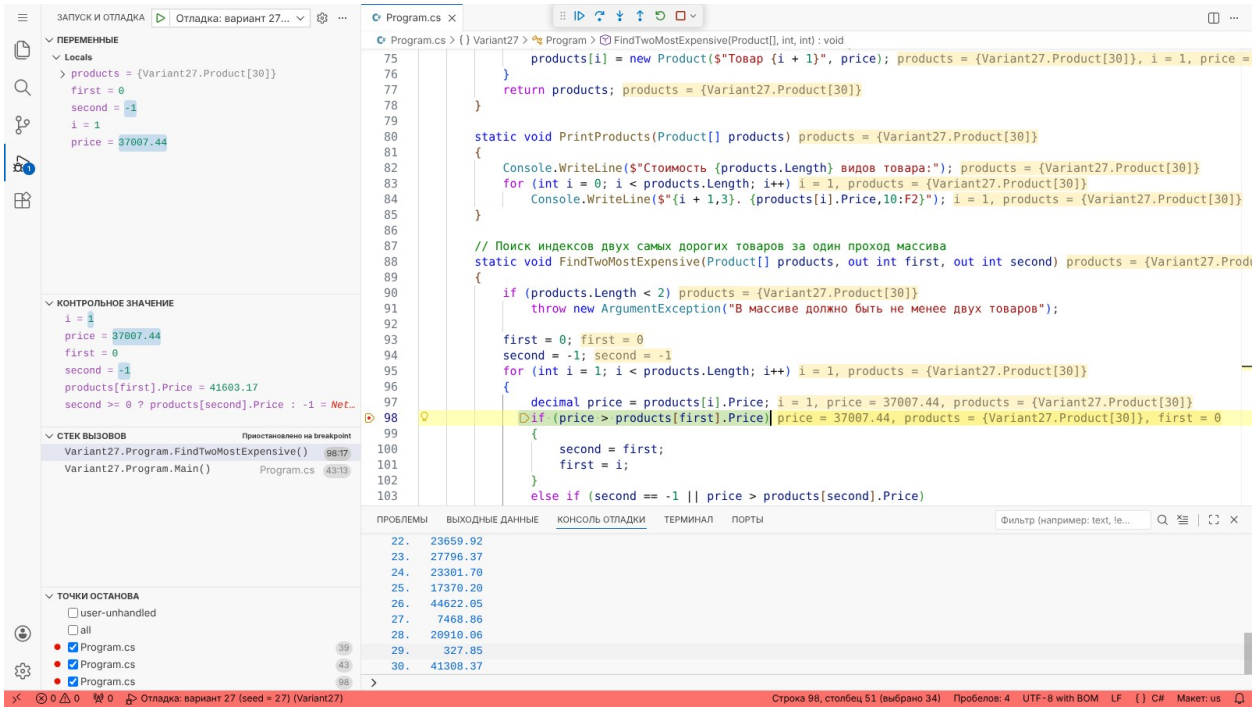


Рисунок 8 – Контрольные значения и стек вызовов на первой итерации поиска

Продолжением выполнения (F5) отладчик доведён до итерации i = 7 (рисунок 9). Контрольные значения показывают, что цена текущего товара price = 49384.08 больше цены текущего максимума products[first].Price =

41603.17, поэтому условие в строке 98 истинно. Изменённые с прошлой остановки значения подсвечиваются.

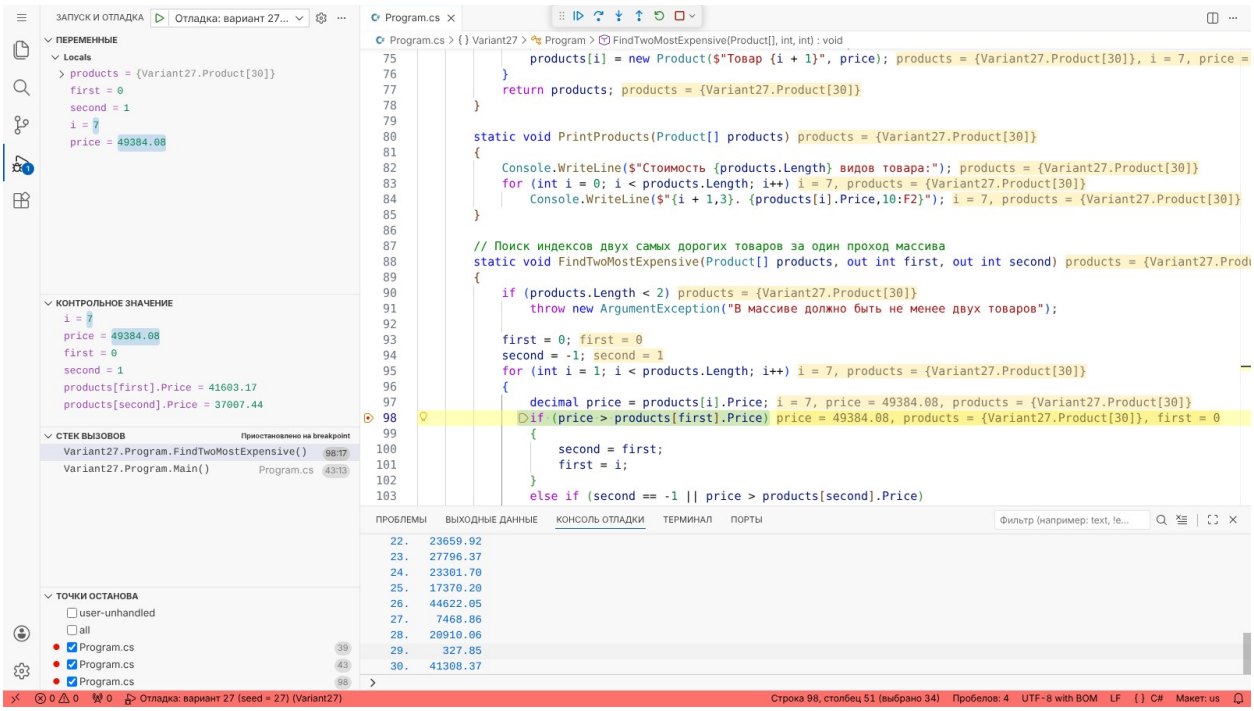


Рисунок 9 – Контрольные значения на итерации i = 7

После двух шагов с обходом (F10) выполнилось присваивание second = first: second = 0, то есть прежний максимум (товар 1) стал вторым по стоимости, и products[second].Price = 41603.17 (рисунок 10). Следующим будет выполнено присваивание first = i. Работа ветки алгоритма соответствует ожидаемой.

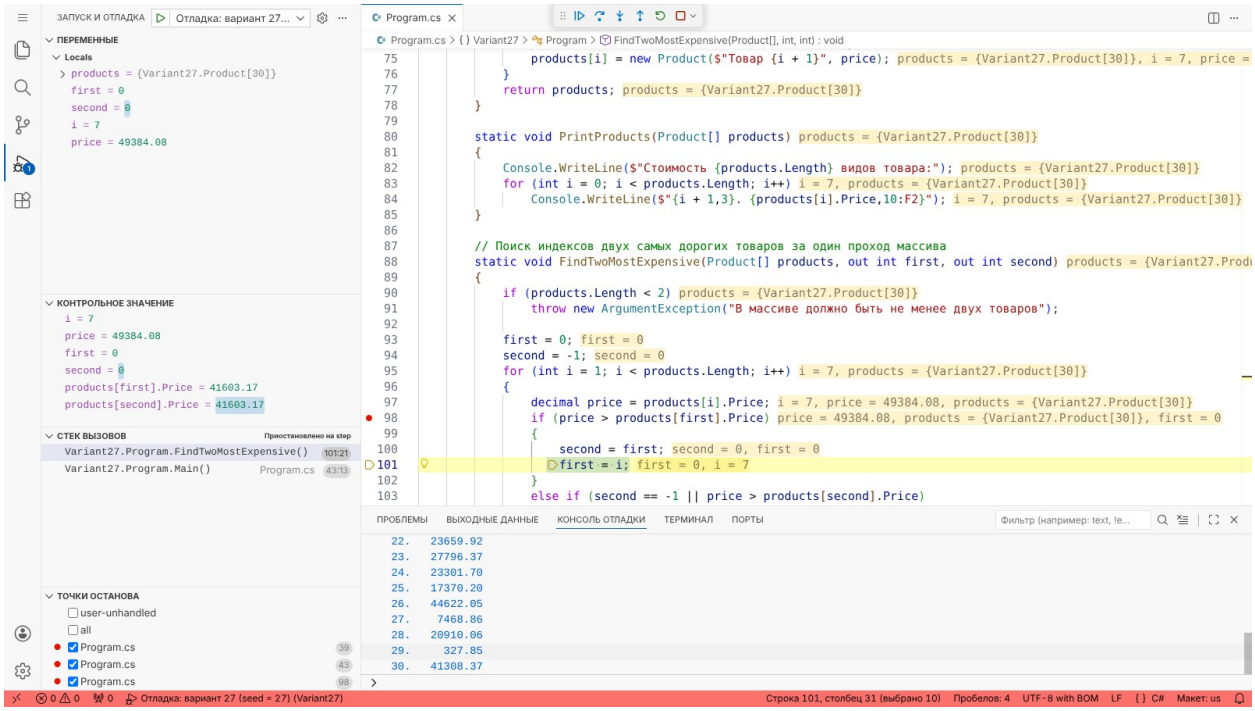


Рисунок 10 – Перенос прежнего максимума во второй максимум

5.7 Просмотр стека вызовов

Окно «Стек вызовов» (рисунок 8) показывает порядок вызова методов: в верхней строке – текущий метод Variant27.Program.FindTwoMostExpensive() и позиция остановки 98:17, во второй – метод Main(), из которого он вызван, с позицией 43:13. Двойной щелчок по строке Main открывает место вызова и переключает окно «Переменные» на локальные переменные метода Main, не перемещая сам отладчик. На рисунке 4 стек вызовов аналогично показывал вызов GenerateProducts из Main, а после шага с выходом в нём остаётся только Main (рисунок 13).

5.8 Условная точка останова

Чтобы не нажимать F5 на каждой из 29 итераций, точка останова в строке 98 настроена под задачу: в контекстном меню точки выбран пункт «Изменить точку останова» и задано условие i == 15 (рисунок 11). Значок точки останова сменился на условный.

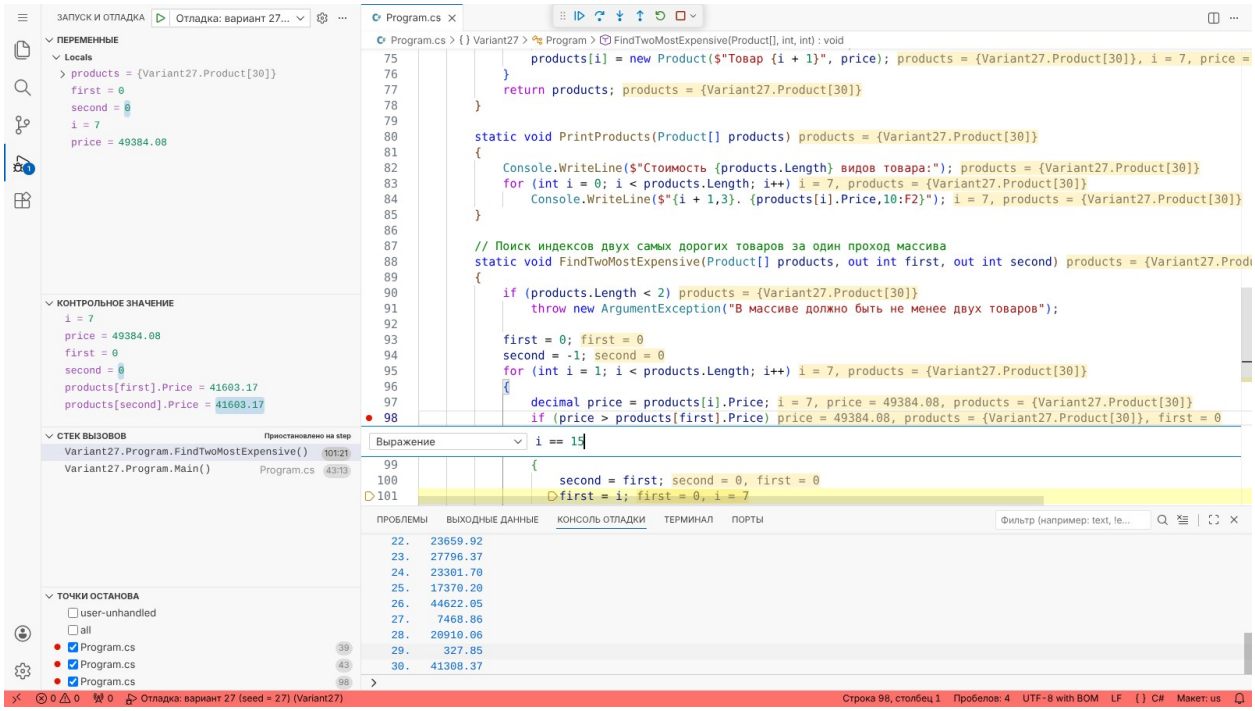


Рисунок 11 – Задание условия для точки останова

После продолжения (F5) отладчик пропустил итерации 8–14 и остановился сразу при i = 15 (рисунок 12). Контрольные значения: first = 7 (максимум 49384.08), second = 14 (44081.45), price = 44632.32. Цена текущего товара меньше максимума, но больше второго значения, поэтому выполнится ветка else if и second станет равным 15.

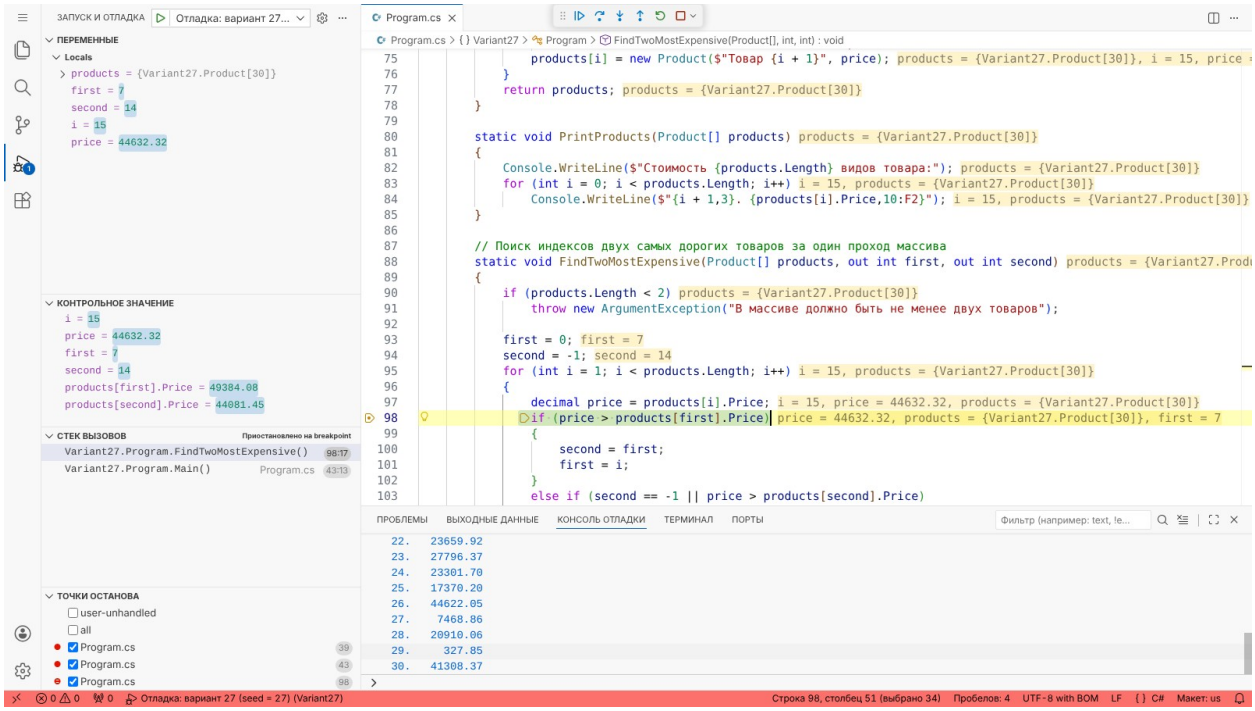


Рисунок 12 – Остановка на условной точке останова при i = 15

5.9 Шаг с выходом и вычисление выражений

Командой «Шаг с выходом» (Shift+F11) метод FindTwoMostExpensive выполнен до конца, и отладчик вернулся в Main (рисунок 13). В окне «Переменные» видны результаты, возвращённые через out-параметры: first = 7, second = 15. Контрольные значения i и price показывают ошибку «The name does not exist in the current context», так как эти переменные существуют только внутри метода поиска, – это наглядно показывает области видимости переменных.

В консоли отладки вычислено выражение products[first].Name + " / " + products[second].Name, результат – "Товар 8 / Товар 16".

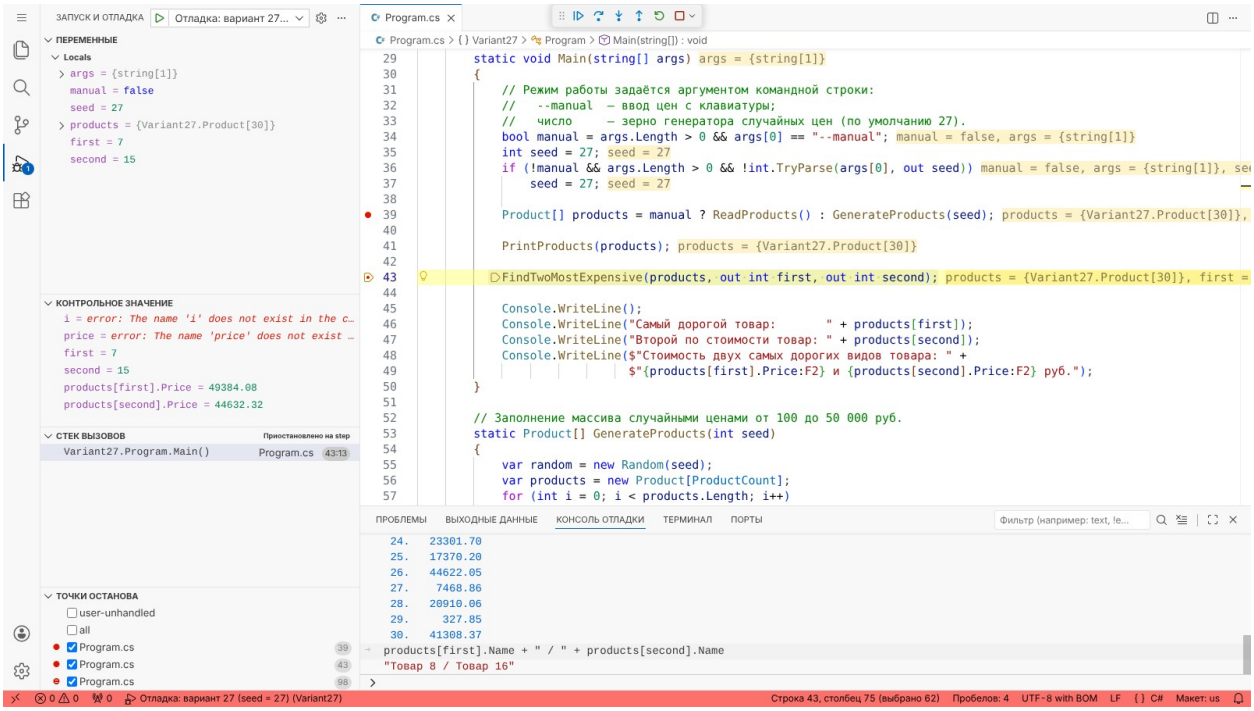


Рисунок 13 – Результат шага с выходом и вычисление выражения в консоли отладки

5.10 Завершение работы программы

После продолжения выполнения (F5) программа завершилась, а в консоли отладки выведен результат (рисунок 14):

Самый дорогой товар: Товар 8 – 49384.08 руб.;

Второй по стоимости товар: Товар 16 – 44632.32 руб.;

Стоимость двух самых дорогих видов товара: 49384.08 и 44632.32 руб.

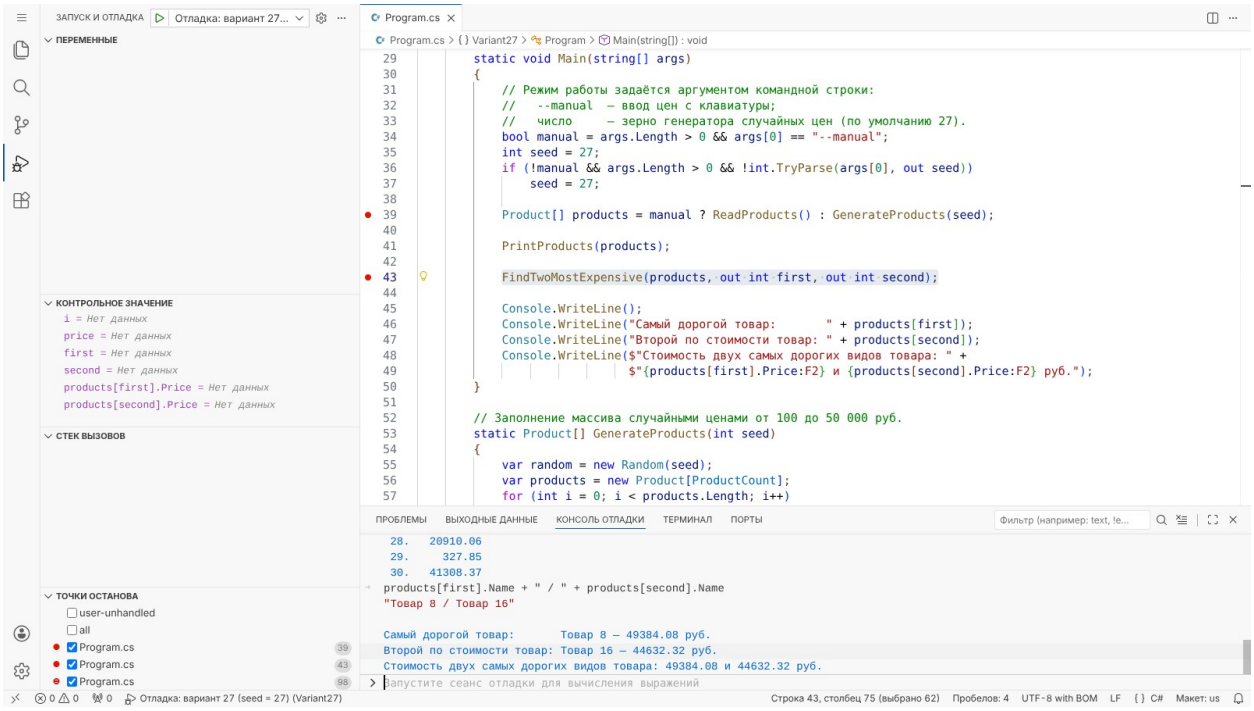


Рисунок 14 – Результат работы программы в консоли отладки

Для контроля программа запущена без отладчика командой dotnet run во встроенном терминале (рисунок 15). Результат совпадает с полученным при отладке.

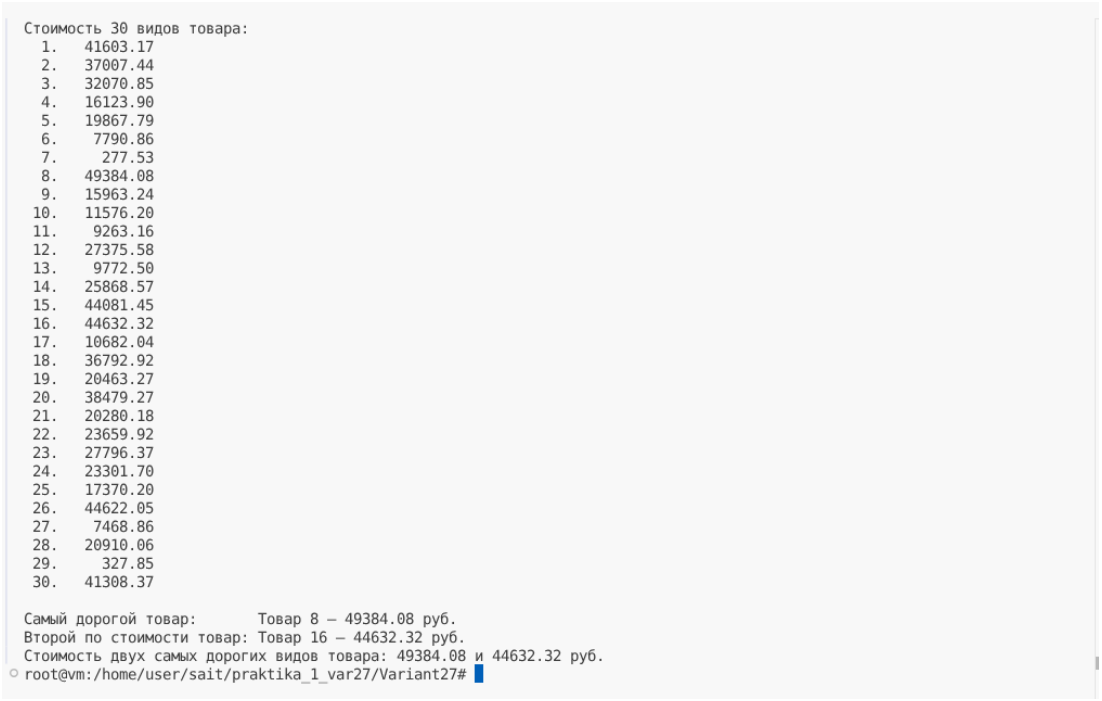


Рисунок 15 – Запуск программы без отладчика

6 Тестирование программы

После отладки программа проверена на наборах данных, приведённых в таблице 2. Ожидаемые значения для случайных массивов получены независимо от программы – сортировкой выведенного списка цен по возрастанию.

Таблица 2 – Результаты тестирования

№	Входные данные	Ожидаемый результат	Фактический результат
1	Аргумент 27 (случайные цены)	49384.08 и 44632.32	49384.08 и 44632.32
2	Аргумент 100 (случайные цены)	49764.47 и 48968.96	49764.47 и 48968.96
3	--manual: 3000, 2900, …, 100 (максимум – первый элемент)	3000.00 и 2900.00	3000.00 и 2900.00
4	--manual: 500, 28 раз по 100, 500 (два одинаковых максимума)	500.00 и 500.00	500.00 и 500.00
5	--manual: «abc», «-5», затем 12.5 и 29 раз по 1	Повторный запрос ввода; 12.50 и 1.00	Дважды «Ошибка ввода…»; 12.50 и 1.00
6	--manual: все 30 цен равны 7	7.00 и 7.00	7.00 и 7.00

Все тесты пройдены, фактические результаты совпадают с ожидаемыми.

ЗАКЛЮЧЕНИЕ

В ходе практического занятия разработана программа на языке C#, определяющая стоимость двух самых дорогих видов товара из 30 (вариант 27), и выполнена её отладка с использованием всех требуемых методов:

– заданы точки останова, в том числе условная точка останова с условием i == 15;

– выполнен пошаговый проход командами «Шаг с заходом» (F11), «Шаг с обходом» (F10) и «Шаг с выходом» (Shift+F11);

– проверены значения переменных и свойств объектов Product (Name, Price) в окне «Переменные» и с помощью подсказок по данным;

– установлены шесть контрольных значений, по которым прослежена работа алгоритма поиска на итерациях 1, 7 и 15;

– просмотрен стек вызовов при заходе в методы GenerateProducts и FindTwoMostExpensive;

– настроена отладка: две конфигурации запуска с аргументами командной строки, автоматическая сборка перед запуском и условная точка останова.

Отладка подтвердила правильность алгоритма: при обнаружении нового максимума прежний максимум переходит во второй, а значения, меньшие максимума, но большие второго, корректно его заменяют. Для исходных данных (зерно 27) получены стоимости 49384.08 и 44632.32 руб. Тестирование на шести наборах данных, включая граничные случаи (одинаковые цены, максимум в первом элементе, некорректный ввод), ошибок не выявило.

ПРИЛОЖЕНИЕ А

(обязательное)

Текст программы

Файл Program.cs

```
 1  // Практическая работа №1 «Отладка программы»
 2  // Вариант 27. В массиве записана информация о стоимости 30 видов товара.
 3  // Определить стоимость двух самых дорогих видов товара.
 4  // Выполнил: Шевырин Александр, группа ПКС-33
 5
 6  using System;
 7
 8  namespace Variant27
 9  {
10      // Вид товара: название и стоимость
11      class Product
12      {
13          public string Name { get; }
14          public decimal Price { get; }
15
16          public Product(string name, decimal price)
17          {
18              Name = name;
19              Price = price;
20          }
21
22          public override string ToString() => $"{Name} — {Price:F2} руб.";
23      }
24
25      class Program
26      {
27          const int ProductCount = 30;
28
29          static void Main(string[] args)
30          {
31              // Режим работы задаётся аргументом командной строки:
32              //   --manual  — ввод цен с клавиатуры;
33              //   число     — зерно генератора случайных цен (по умолчанию 27).
34              bool manual = args.Length > 0 && args[0] == "--manual";
35              int seed = 27;
36              if (!manual && args.Length > 0 && !int.TryParse(args[0], out seed))
37                  seed = 27;
38
39              Product[] products = manual ? ReadProducts() : GenerateProducts(seed);
40
41              PrintProducts(products);
42
43              FindTwoMostExpensive(products, out int first, out int second);
44
45              Console.WriteLine();
46              Console.WriteLine("Самый дорогой товар:       " + products[first]);
47              Console.WriteLine("Второй по стоимости товар: " + products[second]);
48              Console.WriteLine($"Стоимость двух самых дорогих видов товара: " +
49                                $"{products[first].Price:F2} и
{products[second].Price:F2} руб.");
50          }
51
52          // Заполнение массива случайными ценами от 100 до 50 000 руб.
53          static Product[] GenerateProducts(int seed)
54          {
55              var random = new Random(seed);
56              var products = new Product[ProductCount];
57              for (int i = 0; i < products.Length; i++)
58              {
59                  decimal price = random.Next(10000, 5000001) / 100m;
60                  products[i] = new Product($"Товар {i + 1}", price);
61              }
62              return products;
```

```
 63          }
 64
 65          // Ввод цен с клавиатуры с проверкой корректности
 66          static Product[] ReadProducts()
 67          {
 68              var products = new Product[ProductCount];
 69              for (int i = 0; i < products.Length; i++)
 70              {
 71                  decimal price;
 72                  Console.Write($"Стоимость товара {i + 1}: ");
 73                  while (!decimal.TryParse(Console.ReadLine(), out price) || price < 0)
 74                      Console.Write("Ошибка ввода, введите неотрицательное число: ");
 75                  products[i] = new Product($"Товар {i + 1}", price);
 76              }
 77              return products;
 78          }
 79
 80          static void PrintProducts(Product[] products)
 81          {
 82              Console.WriteLine($"Стоимость {products.Length} видов товара:");
 83              for (int i = 0; i < products.Length; i++)
 84                  Console.WriteLine($"{i + 1,3}. {products[i].Price,10:F2}");
 85          }
 86
 87          // Поиск индексов двух самых дорогих товаров за один проход массива
 88          static void FindTwoMostExpensive(Product[] products, out int first, out int
second)
 89          {
 90              if (products.Length < 2)
 91                  throw new ArgumentException("В массиве должно быть не менее двух
товаров");
 92
 93              first = 0;
 94              second = -1;
 95              for (int i = 1; i < products.Length; i++)
 96              {
 97                  decimal price = products[i].Price;
 98                  if (price > products[first].Price)
 99                  {
100                      second = first;
101                      first = i;
102                  }
103                  else if (second == -1 || price > products[second].Price)
104                  {
105                      second = i;
106                  }
107              }
108          }
109      }
110  }
```

ПРИЛОЖЕНИЕ Б

(обязательное)

Файл настройки отладки

Файл .vscode/launch.json

```
{
    // Конфигурации отладки программы (вариант 27)
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Отладка: вариант 27 (seed = 27)",
            "type": "coreclr",
            "request": "launch",
            "preLaunchTask": "build",
            "program": "${workspaceFolder}/bin/Debug/net8.0/Variant27.dll",
            "args": ["27"],
            "cwd": "${workspaceFolder}",
            "console": "internalConsole",
            "stopAtEntry": false,
            "justMyCode": true,
            "env": { "DOTNET_ENVIRONMENT": "Development" }
        },
        {
            "name": "Отладка: ввод цен с клавиатуры",
            "type": "coreclr",
            "request": "launch",
            "preLaunchTask": "build",
            "program": "${workspaceFolder}/bin/Debug/net8.0/Variant27.dll",
            "args": ["--manual"],
            "cwd": "${workspaceFolder}",
            "console": "integratedTerminal",
            "stopAtEntry": true
        }
    ]
}
```

Файл .vscode/tasks.json

```
{
    "version": "2.0.0",
    "tasks": [
        {
            "label": "build",
            "command": "dotnet",
            "type": "process",
            "args": ["build", "${workspaceFolder}/Variant27.csproj", "-c", "Debug"],
            "problemMatcher": "$msCompile"
        }
    ]
}
```